AUTONOMOUS FRAUD DETECTION SYSTEM

Code Documentation

Project: EagleTrust Bank — JATAYU System

Repository: github.com/pragvamsh/Autonomous-Fraud-Detection-System

Generated: March 2026

Table of Contents

Table of Contents 2
1. Project Overview 4
  1.1 System Architecture 4
  1.2 Technology Stack 4
2. Project Structure 6
3. Backend Entry Points 8
  3.1 app.py — Flask Application Entry Point 8
  3.2 db.py — Database Connection & Schema 8
4. API Route Files (et_api/) 10
  4.1 auth_routes.py 10
  4.2 register_routes.py 10
  4.3 payment_routes.py 10
  4.4 account_routes.py 11
  4.5 otp_routes.py 11
  4.6 pattern_routes.py 11
  4.7 raa_routes.py 12
  4.8 aba_routes.py 12
  4.9 cla_routes.py 12
  4.10 admin_routes.py 12
5. TMA — Transaction Monitoring Agent (et_service/monitoring_agent/) 13
  5.1 agent.py — Orchestrator 13
  5.2 anomaly_extractor.py — 15 Feature Signals 13
  5.3 ml_layer.py — Isolation Forest 14
  5.4 decision_engine.py — Score Fusion 14
  5.5 rag/rag_layer.py — 4-Layer RAG Pipeline 15
6. PRA — Pattern Recognition Agent (et_service/pattern_agent/) 16
  6.1 pra_agent.py — Orchestrator & Background Poller 16
  6.2 bilstm_model.py — BiLSTM Architecture 16
  6.3 sequence_builder.py — (30×15) Input Matrix 16
  6.4 pattern_scorer.py — Final Scoring 17
7. RAA — Risk Assessment Agent (et_service/raa/) 18
  7.1 raa_agent.py — Orchestrator 18

7.2 tier_engine.py — Customer Tier Classification 18
7.3 dimension_scorer.py — 5-Dimension Scoring 19
7.4 score_engine.py — Final Score Fusion 19
8. ABA — Alert & Block Agent (et_service/aba/) 20
8.1 aba_agent.py — Orchestrator 20
8.2 gateway_controller.py 20
8.3 notification_engine.py 20
8.4 case_manager.py 20
9. CLA — Citation & Legal Archive Agent (et_service/cla/) 21
9.1 cla_agent.py 21
9.2 str_assembler.py 21
9.3 pdf_generator.py 21
9.4 term_glossary.py 21
10. Data Models (et_model/) 22
11. Data Access Objects (et_dao/) 23
12. Frontend (et-web/) 24
12.1 App.jsx — Router 24
12.2 api.js — Axios Instance 24
12.3 pages/PaymentPage.jsx (1838 lines) 24
12.4 pages/AdminDashboardPage.jsx (714 lines) 25
12.5 pages/RiskEvaluationPage.jsx (768 lines) 25
12.6 Other Pages 25
12.7 Components 25
13. Configuration & Constants 27
13.1 TMA Constants (monitoring_agent/constants.py) 27
13.2 PRA Constants (pattern_agent/constants.py) 27
13.3 CLA Constants (cla/constants.py) 27
14. Testing (et-python/tests/) 29
14.1 Unit Tests 29
14.2 Integration Tests 30
15. Setup & Running 31
15.1 Backend Setup 31
15.2 Frontend Setup 31
15.3 Python Dependencies 31

1. Project Overview

The Autonomous Fraud Detection System, codenamed JATAYU, is a full-stack intelligent banking platform built for EagleTrust Bank. It detects financial fraud in real-time using a multi-agent pipeline powered by machine learning, retrieval-augmented generation (RAG), regulatory rule engines, and a bidirectional LSTM sequence model. The system handles the entire lifecycle from transaction initiation to regulatory filing.

1.1 System Architecture

The system consists of five intelligent agents operating in a sequential pipeline:

Agent	Name	Responsibility
`TMA`	**Transaction Monitoring Agent**	Real-time fraud scoring using Isolation Forest + 4-layer RAG. Returns ALLOW/FLAG/ALERT/BLOCK verdict.
`PRA`	**Pattern Recognition Agent**	Sequence-based fraud pattern detection using BiLSTM (30×15 matrix). Runs asynchronously after TMA.
`RAA`	**Risk Assessment Agent**	Multi-dimensional risk scoring (D1-D5) with 60/40 fusion. Determines regulatory actions.
`ABA`	**Alert & Block Agent**	Executes customer-facing actions: freeze accounts, send notifications, manage fraud cases.
`CLA`	**Citation & Legal Archive Agent**	Generates STR/CTR regulatory filings and archives citations as PDFs.

1.2 Technology Stack

Backend	Python 3.x, Flask 3.0.2, Flask-CORS 4.0.0
Database	MySQL (mysql-connector-python 8.3.0)
ML Models	Isolation Forest (scikit-learn 1.5.1), BiLSTM (PyTorch)
Vector DB	ChromaDB >= 0.4.0 (3 collections: L1 Regulatory, L2 Fraud Cases, L3 Typologies)
Embeddings	SentenceTransformer all-MiniLM-L6-v2
PDF Generation	ReportLab

Frontend	React 18 + Vite 5, Material UI 5, Axios, React Router 6
Auth	Flask sessions + bcrypt password hashing
Testing	pytest, pytest-cov

2. Project Structure

```
Autonomous-Fraud-Detection-System/
├── requirements.txt                # Python dependencies
├── models/
│   └── bilstm_v1.pt                # Pre-trained BiLSTM model
├── et-python/                      # Backend (Flask)
│   ├── app.py                      # Flask app + startup bootstrap
│   ├── db.py                       # DB connection + schema creation
│   ├── et_api/                     # API route blueprints
│   │   ├── auth_routes.py
│   │   ├── register_routes.py
│   │   ├── account_routes.py
│   │   ├── otp_routes.py
│   │   ├── payment_routes.py
│   │   ├── pattern_routes.py
│   │   ├── raa_routes.py
│   │   ├── aba_routes.py
│   │   ├── cla_routes.py
│   │   └── admin_routes.py
│   ├── et_dao/                     # Data Access Objects
│   │   ├── auth_dao.py
│   │   ├── account_dao.py
│   │   ├── customer_dao.py
│   │   ├── monitoring_dao.py
│   │   ├── otp_dao.py
│   │   ├── payment_dao.py
│   │   ├── pattern_dao.py
│   │   ├── raa_dao.py
│   │   ├── aba_dao.py
│   │   └── cla_dao.py
│   ├── et_model/                   # Data models / DTOs
│   │   ├── customer.py
│   │   ├── fraud_alert.py
│   │   ├── otp.py
│   │   ├── pattern_alert.py
│   │   └── raa_action_package.py
│   ├── et_service/                 # Business logic + agents
│   │   ├── auth_service.py
│   │   ├── account_service.py
│   │   ├── payment_service.py
│   │   ├── register_service.py
│   │   ├── otp_service.py
│   │   ├── monitoring_agent/       # TMA — Transaction Monitoring Agent
│   │   │   ├── agent.py
│   │   │   ├── anomaly_extractor.py
│   │   │   ├── profile_builder.py
│   │   │   ├── ml_layer.py
│   │   │   ├── decision_engine.py
│   │   │   ├── response_executor.py
│   │   │   ├── constants.py
│   │   │   ├── rag/                # RAG layer
│   │   │   │   ├── rag_layer.py
│   │   │   │   ├── encoders.py
```

```
│   │   │   │   ├── vector_store.py
│   │   │   │   └── kb_ingest/
│   │   │   └── training/
│   │   ├── pattern_agent/          # PRA — Pattern Recognition Agent
│   │   │   ├── pra_agent.py
│   │   │   ├── bilstm_model.py
│   │   │   ├── sequence_builder.py
│   │   │   ├── pattern_scorer.py
│   │   │   ├── pra_rag_layer.py
│   │   │   ├── train_bilstm.py
│   │   │   └── constants.py
│   │   ├── raa/                    # RAA — Risk Assessment Agent
│   │   │   ├── raa_agent.py
│   │   │   ├── intelligence_aggregator.py
│   │   │   ├── tier_engine.py
│   │   │   ├── dimension_scorer.py
│   │   │   ├── raa_rag_layer.py
│   │   │   ├── score_engine.py
│   │   │   ├── regulatory_engine.py
│   │   │   └── action_package_builder.py
│   │   ├── aba/                    # ABA — Alert & Block Agent
│   │   │   ├── aba_agent.py
│   │   │   ├── gateway_controller.py
│   │   │   ├── action_executor.py
│   │   │   ├── notification_engine.py
│   │   │   ├── case_manager.py
│   │   │   └── constants.py
│   │   └── cla/                    # CLA — Citation & Legal Archive
│   │       ├── cla_agent.py
│   │       ├── citation_archiver.py
│   │       ├── str_assembler.py
│   │       ├── pdf_generator.py
│   │       ├── term_glossary.py
│   │       └── constants.py
│   └── tests/                      # Unit + integration tests
└── et-web/                         # Frontend (React + Vite)
    ├── src/
    │   ├── App.jsx
    │   ├── api.js
    │   ├── main.jsx
    │   ├── theme.js
    │   ├── components/
    │   └── pages/
    ├── index.html
    └── package.json
```

3. Backend Entry Points

3.1 app.py — Flask Application Entry Point

The main Flask application file. Configures CORS, registers all API blueprints, bootstraps the database schema, and sequentially initializes all agents and ML models on startup.

Startup Bootstrap Sequence (7 steps, synchronous):

1. Pre-load SentenceTransformer BERT model (all-MiniLM-L6-v2)
2. ChromaDB health check — auto-rebuilds if corrupted (RustBindingsAPI error)
3. Isolation Forest model check — auto-trains if missing

3b. BiLSTM model warning if file not found

4. PRA background poller thread (500ms interval)
5. RAA background poller thread (500ms interval)
6. ABA background poller thread (500ms interval)
7. CLA background poller thread (500ms interval)

```
# app.py — Key code
app = Flask(__name__)
app.config['SECRET_KEY'] = secrets.token_hex(32)
app.config['PERMANENT_SESSION_LIFETIME'] = 3600 * 8  # 8 hours

# CORS — allow frontend on ports 5173 and 5174
CORS(app, resources={
    r"/api/*": {
        "origins": ["http://localhost:5173", "http://localhost:5174"],
        "supports_credentials": True,
    }
})

# Blueprint registration
app.register_blueprint(payment_bp, url_prefix='/api')
app.register_blueprint(raa_bp,     url_prefix='/api')
# ... (all 10 blueprints registered)
```

3.2 db.py — Database Connection & Schema

Manages the MySQL connection and auto-creates all database tables on startup. Uses idempotent CREATE TABLE IF NOT EXISTS and ALTER TABLE migration statements via a _safe_alter() helper that silently ignores duplicate column errors (errno 1060).

Database Tables Created:

Table	Description
`customers`	Customer accounts: PII, KYC fields, account number, balance, frozen status
`otp_tokens`	OTP tokens for email verification, password change, and FRAUD_MFA
`transactions`	All debit/credit transactions with risk_score, fraud_flag, agent_status
`customer_behaviour_profiles`	90-day behavioural baseline: avg/std/max amounts, daily volume, usual hours
`fraud_alerts`	TMA + PRA + RAA results: scores, verdicts, citations, reasoning, RAA stages JSON
`pattern_alerts`	PRA-generated pattern alerts linked to payments
`action_packages`	RAA-dispatched action packages consumed by ABA
`fraud_cases`	Fraud cases created by ABA's case_manager, consumed by CLA
`regulatory_queue`	STR/CTR filings with status: AUTO_FILED, PENDING_APPROVAL, REJECTED
`cla_archive`	CLA citation archives with PDF paths
`aba_execution_log`	Audit log of all ABA actions executed

```
# db.py — DB Config
DB_CONFIG = {
    'host': '127.0.0.1',
    'user': 'root',
    'password': 'Enter Your Password',  # Change before running
    'database': 'fraud_detection_test',
    'auth_plugin': 'mysql_native_password'
}
```

4. API Route Files (et_api/)

All routes are Flask Blueprints. Most are registered under the /api prefix in app.py. Session-based authentication is enforced by decorators: @login_required (customer) and @admin_required (admin).

4.1 auth_routes.py

Blueprint: auth_bp | URL Prefix: /api

Endpoint	Method	Description
`/api/login`	**POST**	Authenticate with customer ID/email + password. Returns session cookie.
`/api/me`	**GET**	Returns authenticated customer profile.
`/api/logout`	**POST**	Clears session cookie.
`/api/set-password`	**POST**	Set initial password after first login (requires OTP verification).
`/api/change-password`	**POST**	Change password (requires current password + OTP).
`/api/update-email`	**PUT**	Update email address.
`/api/update-phone`	**PUT**	Update phone number.

4.2 register_routes.py

Blueprint: register_bp | URL Prefix: /api

Endpoint: POST /api/register
Validates registration data, creates a CustomerRegistration model, inserts into DB, and sets session. Returns customer_id and account_number.

4.3 payment_routes.py

Blueprint: payment_bp | URL Prefix: /api | Requires: @login_required

This is the most complex route file. The POST /api/payment endpoint implements the full fraud-check gate:

1. Validate input — amount, recipient account, sender balance
2. process_payment() — creates PENDING payment record in DB
3. evaluate_transaction() (TMA) — synchronous fraud scoring
4. Route based on TMA score:
   - Score 0-44 (ALLOW/FLAG): Commit immediately, fire PRA in background thread
   - Score 45-75 (FLAG+MFA/ALERT): Hold payment, send OTP, return HTTP 202 for frontend OTP screen
   - Score 76-100 (BLOCK): Reject payment, return HTTP 403

```
# payment_routes.py — Core flow logic
@payment_bp.route('/payment', methods=['POST'])
@login_required
def make_payment(customer_id):
    # ... validate, process, evaluate ...
    if tma_score >= 76:
        reject_payment(payment_id)   # BLOCK
        return jsonify({...}), 403
    elif tma_score >= 45:
        hold_payment(payment_id)     # FLAG+MFA / ALERT
        send_otp_for_fraud_mfa(...)
        return jsonify({...}), 202   # Frontend shows OTP screen
    else:
        commit_payment(payment_id)   # ALLOW / FLAG
        threading.Thread(target=_fire_pra, ...).start()
        return jsonify({...}), 200
```

4.4 account_routes.py

Blueprint: account_bp | URL Prefix: /api
Endpoints: GET /api/account (balance + profile), POST /api/account/add-money (top-up), GET /api/transactions (history, max 100).

4.5 otp_routes.py

Blueprint: otp_bp | URL Prefix: /api
Endpoints: POST /api/send-otp (EMAIL_VERIFY or PASSWORD_CHANGE), POST /api/verify-email-otp, POST /api/change-password-otp. Rate-limited via ValueError from otp_dao.

4.6 pattern_routes.py

Blueprint: pattern_bp | URL Prefix: /api
GET /api/pattern-profile — returns rolling BiLSTM pattern profile with trend, escalation count, and rolling avg risk. GET /api/pattern-alert/<payment_id> — polls PRA result for a specific payment.

4.7 raa_routes.py

Blueprint: raa_bp | URL Prefix: /api
GET /api/raa/health (no auth), GET /api/raa/alerts/<payment_id> (returns 202 while processing, 200 when raa_processed=1 with full verdict + stage progress JSON).

4.8 aba_routes.py

Blueprint: aba_bp | URL Prefix: /api
GET /api/aba/health (queue stats), GET /api/aba/case/<case_id> (case details for authenticated customer), POST /api/aba/confirm/<payment_id> (customer confirms FLAG transaction), POST /api/aba/verify-otp (OTP verification for held payments).

4.9 cla_routes.py

Blueprint: cla_bp | All endpoints under /api/cla/ | @admin_required
Provides: GET /api/cla/filings, GET /api/cla/filing/<id>, GET /api/cla/cases/<case_id>, POST /api/cla/export-pdf/<filing_id> (generates PDF on-demand using ReportLab), POST /api/cla/process-case/<case_id> (manual re-processing). Falls back to generating PDF directly from fraud_cases if CLA has not yet processed the case.

4.10 admin_routes.py

Blueprint: admin_bp | No URL prefix (admin routes use /api/admin/*)
Admin credentials are stored in et_service/cla/constants.py. Session key admin_logged_in is completely isolated from customer session key customer_id.
Endpoints: POST /api/admin/login, POST /api/admin/logout, GET /api/admin/check_auth, GET /api/admin/dashboard (system-wide stats), GET /api/admin/filings (FLAG/ALERT/BLOCK alerts), POST /api/admin/unfreeze/<customer_id>, GET /api/admin/cases (open fraud cases).

5. TMA — Transaction Monitoring Agent (et_service/monitoring_agent/)

The TMA is the first and most critical agent. It evaluates every payment in real-time using a 6-stage pipeline before the transaction response is returned to the customer.

5.1 agent.py — Orchestrator

Entry point: evaluate_transaction(payment_result: dict). The pipeline runs synchronously in the HTTP request thread. Failures in stages 3 and 4 (ML and RAG) do NOT abort — the pipeline degrades gracefully to rule-based or ML-only mode.

6-Stage Pipeline:

Stage	Module	Description
1	`profile_builder.py`	Build/fetch 90-day behavioural baseline (avg, std, max amounts, daily volume, usual hours). Cached for 4 hours.
2	`anomaly_extractor.py`	Extract 15 anomaly signals (amount z-score, velocity, new recipient, cyclical hour encoding, structuring bands, etc.)
3	`ml_layer.py`	Isolation Forest: 15 features → 0-100 ML risk score. Fallback to rule-based scoring if model missing.
4	`rag/rag_layer.py`	4-Layer ChromaDB retrieval: L2 fraud cases, L1 regulatory rules, L3 typologies, L4 dynamic weights.
5	`decision_engine.py`	Score fusion (ML 40% + RAG 60%). Applies confidence threshold, disagreement detection, cold-start penalty.
6	`response_executor.py`	Writes fraud_alert row to DB. Updates transaction agent_status to EVALUATED.

5.2 anomaly_extractor.py — 15 Feature Signals

Derives 15 anomaly features fed to both the Isolation Forest and the RAG query encoders.

Feature	Type	Description
`amount_z_score`	Numeric	Standard deviations from personal mean amount
`amount_vs_max`	Numeric	Ratio to personal historical maximum

`is_large_amount`	Binary	Amount > 10× personal average
`is_new_recipient`	Binary	Recipient not in known recipients list
`is_velocity_burst`	Binary	More than 3 transactions in last 1 hour
`transactions_last_1h`	Count	Number of transactions in last 1 hour
`transactions_last_24h`	Count	Number of transactions in last 24 hours
`exceeds_daily_volume`	Binary	Today's total > 2× 90-day average daily volume
`is_near_threshold`	Binary	Amount in structuring band (just below KYC/EDD thresholds)
`is_round_number`	Binary	Amount is multiple of 1000 and >= Rs 5000
`is_unusual_hour`	Binary	Transaction at unusual hours for this customer
`high_z_new_recipient`	Binary	Combination: z-score > 2 AND new recipient
`hour_sin`	Numeric	Sine of current hour (cyclical encoding, avoids midnight discontinuity)
`hour_cos`	Numeric	Cosine of current hour (cyclical encoding)
`amount_vs_daily_avg`	Numeric	Amount vs 90-day average daily volume

5.3 ml_layer.py — Isolation Forest

Loads a pre-trained Isolation Forest pipeline (StandardScaler + IsolationForest) from models/isolation_forest.pkl. The model file must contain a dict with model and calibration keys. Score normalization uses training-derived min/max from the calibration dict for linear inversion to 0-100 risk score.

Fallback behavior (model missing): Rule-based scoring with FLAG floor (31) as baseline, adding bonuses for high z-score, new recipient, velocity burst, daily volume excess, structuring, and high_z_new_recipient.

5.4 decision_engine.py — Score Fusion

Fuses ML and RAG scores using the following priority logic:
Mode 1: RAG unavailable → ML score only
Mode 2: Confidence < 0.65 → ML score only, floored at 31 (FLAG minimum)
Mode 3: |ML - RAG| > 30 → Conservative max score (disagreement flag set)
Mode 4: Normal → Weighted average using L4 dynamic weights (default ML=40%, RAG=60%)
Post-fusion: Cold start penalty (+COLD_START_PENALTY) applied

Decision Tiers:

0-25	ALLOW — Transaction proceeds
26-44	FLAG — Proceed, PRA notified
45-75	ALERT — Hold, OTP required
76-100	BLOCK — Transaction rejected

5.5 rag/rag_layer.py — 4-Layer RAG Pipeline

Queries ChromaDB using vector and text similarity across three knowledge base collections:

L2 fraud_cases	Vector query using FraudFeatureEncoder (128-d). Returns similarity-weighted confirmed_risk_score and confidence.
L1 regulatory	Text query using natural language regulatory context. Returns RBI/PMLA rule matches and adjustment score.
L3 typologies	Vector query using SignalSequenceEncoder (256-d). Returns FIU-IND typology match (e.g. TY-07, TY-12) and adjustment.
L4 dynamic weights	Reads accuracy history from DB to compute dynamic ML/RAG fusion weights for the decision engine.

6. PRA — Pattern Recognition Agent (et_service/pattern_agent/)

The PRA runs asynchronously after TMA. It polls fraud_alerts every 500ms for FLAG/ALERT/BLOCK rows where pra_processed=0. Uses atomic claiming (pra_processed=2) to prevent duplicate processing across worker threads.

6.1 pra_agent.py — Orchestrator & Background Poller

5-Stage Pipeline per alert:
Stage 1: Read TMA fraud_alerts row
Stage 2: sequence_builder → (30×15) input matrix
Stage 3: BiLSTM inference → bilstm_score (0-100) + hidden_state (128-d)
Stage 4: 3-Layer RAG → urgency_multiplier, precedent_adj, reg_adj
Stage 5: pattern_scorer → final_pattern_score + pra_verdict

PRA Verdicts: NORMAL / ELEVATED / HIGH / CRITICAL

6.2 bilstm_model.py — BiLSTM Architecture

Defines FraudBiLSTM: a 2-layer bidirectional LSTM for sequence classification. Model weights are loaded from models/bilstm_v1.pt.

```
# FraudBiLSTM Architecture
Input:  (batch, 30, 15)     # 30 transactions × 15 features
Layer1: BiLSTM(15 → 128)    # 64 per direction, bidirectional
Dropout: 0.3
Layer2: BiLSTM(128 → 128)
Last h:  h2[:, -1, :]       # shape (batch, 128)
FC1:    Linear(128 → 64) + ReLU
FC2:    Linear(64 → 1) + sigmoid × 100

Outputs:
  bilstm_score: float 0-100 (fraud probability)
  hidden_state: Tensor (batch, 128) — used as RAG query vector
```

6.3 sequence_builder.py — (30×15) Input Matrix

Builds the input sequence matrix from the customer's transaction history. For each of the last 30 DEBIT transactions, it either uses the feature_snapshot stored in fraud_alerts by TMA (preferred) or re-extracts features from the raw transaction. Zero-pads rows for customers with fewer than 30 transactions.

6.4 pattern_scorer.py — Final Scoring

Computes final_pattern_score from: bilstm_score, precedent_adj (L2 case similarity), reg_adj (L1 regulatory adjustment), and urgency_multiplier (L3 typology match). Maps score to verdict: NORMAL (<40), ELEVATED (40-59), HIGH (60-79), CRITICAL (>=80).

7. RAA — Risk Assessment Agent (et_service/raa/)

The RAA polls fraud_alerts where pra_processed=1 AND raa_processed=0. It runs an 8-stage pipeline to produce a final risk score and verdict, then dispatches an action package to ABA. Stage progress is written to raa_stages JSON column in real-time.

7.1 raa_agent.py — Orchestrator

8-Stage Pipeline:

Stage	Module	Description
1	`intelligence_aggregator.py`	Reads and validates the fraud_alerts row. Validates required fields.
2	`tier_engine.py`	Classifies customer into T1-T4 tier. CRITICAL pra_verdict forces T1 demotion.
3	`dimension_scorer.py`	Scores 5 dimensions (D1-D5) using tier-specific weights. Produces Score_A.
4	`raa_rag_layer.py`	L2 fraud cases, L1 regulatory, L3 typology retrievals. Returns pattern_mult, regulatory_adj.
5	`score_engine.py`	60/40 fusion (Score_B × 0.60 + Score_A_adj × 0.40). Applies tier floors. Produces final_raa_score.
6	`regulatory_engine.py`	Checks CTR threshold (Rs 10 lakh) and STR obligation. Auto-drafts STR if required.
7	`action_package_builder.py`	Assembles action package dict and inserts into action_packages table for ABA.
8	`DB Write-back`	Sets raa_processed=1 in fraud_alerts. Always runs in finally block.

7.2 tier_engine.py — Customer Tier Classification

Classifies customers using AND logic into four tiers. The CRITICAL pra_verdict forces T1 regardless of history.

T1 — Highest Risk	tx_count < 15 AND account_age < 14 days AND any historical fraud flag

T2 — High Risk	tx_count >= 15 AND account_age >= 14 days AND zero flags ever (default)
T3 — Mature	tx_count >= 50 AND account_age >= 60 days AND zero flags in last 30d
T4 — Veteran	tx_count >= 200 AND account_age >= 180 days AND zero flags in last 90d

7.3 dimension_scorer.py — 5-Dimension Scoring

Each dimension scores 0-100 using formulas from the spec. Tier-specific weights are applied to produce Score_A.

D1 — Transaction Anomaly	40% latest TMA score + 30% avg ML score + 20% flag count + 10% trend slope
D2 — Behavioural Pattern	30% amount z-score + 25% frequency + 25% new recipient rate + 20% amount entropy
D3 — Network Risk	35% graph risk (PRA pattern_score) + 30% fraud network % + 20% velocity + 15% connections
D4 — Identity Trust	Account age, security score, KYC, profile changes, minor flag
D5 — Temporal Suspicion	Time gaps, hour anomaly, high-risk period, session timing

7.4 score_engine.py — Final Score Fusion

Fuses Score_A and Score_B with 60/40 weighting. Score_B is the raw transaction danger score computed from amount z-score, new recipient flag, velocity, and unusual hour. Applies tier floors (T1=25, T2=20, T3=10, T4=0) and CRITICAL floor=60.

RAA Verdicts: ALLOW (0-25) / FLAG (26-50) / ALERT (51-75) / BLOCK (76-100)

8. ABA — Alert & Block Agent (et_service/aba/)

The ABA consumes action_packages from RAA and executes all customer-facing actions. It polls action_packages where aba_consumed=0. Stale claims (aba_consumed=2 for > 120 seconds) are auto-recovered on startup.

8.1 aba_agent.py — Orchestrator

7-Stage Pipeline per action package:
Stage 1 — Package Loader: Reads and validates action_packages row
Stage 2 — Gateway Controller: Determines APPROVE / HELD / STOPPED
Stage 3 — Action Executor: Dispatches verdict-specific actions (freeze account, initiate reversal)
Stage 4 — Notification Dispatcher: Push/email/SMS to customer
Stage 5 — Account Controller: Freeze/unfreeze/credential reset
Stage 6 — Regulatory Router: CTR/STR queue insertion
Stage 7 — Case Manager: Creates fraud_case record, hands off to CLA

8.2 gateway_controller.py

Maps RAA verdict to gateway action: ALLOW → APPROVE, FLAG → APPROVE with alert, ALERT → HELD, BLOCK → STOPPED.

8.3 notification_engine.py

Sends in-app, email, and SMS notifications based on verdict severity. Notification content is personalized per customer and includes fraud typology description.

8.4 case_manager.py

Creates fraud_cases rows for ALERT and BLOCK verdicts. Queues STR/CTR regulatory filings into regulatory_queue. Links cases to CLA via cla_consumed=0 flag.

9. CLA — Citation & Legal Archive Agent (et_service/cla/)

The CLA is the final agent. It polls fraud_cases where cla_consumed=0, assembles regulatory documents (STR/CTR), archives citations, and manages the regulatory filing lifecycle.

9.1 cla_agent.py

7-Step per case: (1) Retrieve case + alert + transaction + customer data. (2) Archive citations from TMA, RAA, and PRA rag_citations fields. (3) Determine STR vs CTR based on amount threshold (Rs 10 lakh = CTR). (4) Determine filing status from RAA score: >=85=AUTO_FILED, >=70=PENDING_APPROVAL, <70=REJECTED. (5) Insert into regulatory_queue. (6) Archive in cla_archive. (7) Mark case consumed.

9.2 str_assembler.py

Assembles a structured STR JSON document from case, alert, transaction, customer data and citation IDs. Includes: filing_id, narrative, typology_code, severity (from RAA score), citations, and investigation_note.

9.3 pdf_generator.py

Generates formatted A4 PDF reports using ReportLab. Uses EagleTrust brand colors (brown/cream theme). Produces STR and CTR PDF variants. Falls back gracefully if ReportLab is not installed. PDFs are saved to cla_archive/pdfs/ directory.

9.4 term_glossary.py

Translates technical system codes into human-readable terms for inclusion in PDF reports. Translates anomaly_flag codes, typology codes, and verdict codes into plain English explanations.

10. Data Models (et_model/)

File	Description
`customer.py`	CustomerRegistration dataclass: validates PAN, Aadhaar (hashed), DOB, email, phone, account_type. from_dict() factory method.
`fraud_alert.py`	FraudAlert dataclass: holds all TMA outputs — risk_score, ml_score, rag_score, decision, anomaly_flags, rag_citations, agent_reasoning, typology_code, disagreement flags.
`otp.py`	OTP model with generation, hashing, and expiry logic.
`pattern_alert.py`	PatternAlert dataclass: holds PRA outputs — pattern_score, bilstm_score, pra_verdict, urgency_multiplier, sequence_length.
`raa_action_package.py`	ActionPackage dataclass: holds the full RAA output payload dispatched to ABA — verdict, tier, dimensions, regulatory flags, STR draft.

11. Data Access Objects (et_dao/)

All database operations are encapsulated in DAO modules. Each module handles one entity domain. Uses mysql-connector-python with explicit cursor management and connection cleanup.

DAO File	Key Functions
`auth_dao.py`	find_customer_by_email_or_id(), find_customer_by_id(), set_password(), mark_email_verified()
`account_dao.py`	get_balance(), credit_account(), debit_account(), get_transactions()
`customer_dao.py`	insert_customer(), get_customer_by_id(), update_customer_fields()
`monitoring_dao.py`	update_transaction_after_evaluation(), get_transactions_last_n_hours(), get_behaviour_profile(), upsert_behaviour_profile(), get_alert_by_id(), get_fraud_alert_by_payment()
`otp_dao.py`	store_otp() (rate-limited), fetch_latest_otp(), mark_otp_used()
`payment_dao.py`	create_payment(), commit_payment(), hold_payment(), reject_payment(), get_held_payment()
`pattern_dao.py`	get_unprocessed_alerts(), claim_single_alert() (atomic), write_pra_result(), save_pattern_alert(), get_pattern_profile()
`raa_dao.py`	get_unprocessed_alerts() (pra=1 AND raa=0), mark_raa_processed(), mark_raa_processed_flag(), insert_regulatory_filing(), get_raa_health_stats()
`aba_dao.py`	get_unconsumed_packages(), mark_aba_consumed(), update_fraud_alert_aba(), save_execution_log(), get_fraud_case(), recover_stale_aba_claims()
`cla_dao.py`	get_unclaimed_cases(), mark_case_consumed(), insert_archive(), get_all_filings(), update_filing_status(), get_dashboard_stats()

12. Frontend (et-web/)

React 18 single-page application built with Vite 5. Uses Material UI 5 for components and maintains a brown/cream brand theme throughout. All API calls go through the central api.js Axios instance.

12.1 App.jsx — Router

```
// Route structure
/               → HomePage
/login          → LoginPage
/register       → RegisterPage
/customer-dashboard (alias /dashboard) → CustomerDashboardPage
/profile        → UpdateProfilePage
/payment        → PaymentPage
/risk           → RiskEvaluationPage
/transactions   → TransactionHistoryPage
/goadmin        → AdminLoginPage
/admin/dashboard → AdminDashboardPage
```

12.2 api.js — Axios Instance

Configures Axios with baseURL=http://localhost:5000/api, withCredentials=true (cookies), and a global 401 interceptor that redirects to /login on session expiry.

12.3 pages/PaymentPage.jsx (1838 lines)

The most complex frontend page. Handles:

- Payment form with recipient account and amount validation
- TMA pipeline animation: 7-step progress stepper showing each agent stage
- OTP screen for FLAG+MFA/ALERT verdicts (HELD payments)
- PRA result polling with BiLSTM score display
- RAA result polling with stage-by-stage progress display
- Verdict modals: FlagConfirmationModal and BlockNotificationModal
- Account frozen detection and banner

12.4 pages/AdminDashboardPage.jsx (714 lines)

Full admin dashboard with tabs for: Overview stats, Flagged filings (FLAG/ALERT/BLOCK alerts), Fraud cases, Regulatory filings (STR/CTR), Account unfreezing controls. Displays live verdict counts, agent health stats, and a live pipeline activity feed.

12.5 pages/RiskEvaluationPage.jsx (768 lines)

Customer-facing risk profile page. Displays: current customer tier (T1-T4), recent fraud alert history with TMA/PRA/RAA scores, BiLSTM pattern profile with trend direction, rolling average risk, and escalation count. Calls GET /api/raa/customer-alerts and GET /api/pattern-profile.

12.6 Other Pages

Page	Description
`LoginPage.jsx`	Login form with customer ID or email + password. Detects frozen account on response and warns.
`RegisterPage.jsx`	Multi-step registration form: personal info, KYC (PAN, Aadhaar), address, account type.
`CustomerDashboardPage.jsx`	Account balance, recent transactions, quick links to payment and risk pages.
`UpdateProfilePage.jsx`	Update email and phone number with OTP verification.
`TransactionHistoryPage.jsx`	Paginated transaction history with risk score badges, fraud flags, and filter controls.
`AdminLoginPage.jsx`	Admin-only login form. Posts to /api/admin/login.

12.7 Components

Component	Description
`AccountFrozenBanner.jsx`	Banner shown on payment page when account is frozen due to BLOCK verdict.
`FlagConfirmationModal.jsx`	Modal for customer to confirm or dispute a FLAG transaction.
`FraudCarousel.jsx`	Home page carousel educating users about common fraud types.
`HeroSection.jsx`	Hero section for HomePage with bank branding.
`Navbar.jsx`	Top navigation bar with customer name, account balance, and navigation links.
`QuickLinksSection.jsx`	Quick links grid for common customer actions.
`ServicesSection.jsx`	Services overview section for HomePage.

`VideosSection.jsx`	Educational videos section for HomePage.
`Footer.jsx`	Site footer with bank info and links.

13. Configuration & Constants

All magic numbers are centralized in constants.py files within each agent module. Key constants:

13.1 TMA Constants (monitoring_agent/constants.py)

CTR_THRESHOLD	Rs 10,00,000 — Cash Transaction Report threshold (RBI/FIU-IND)
EDD_THRESHOLD	Rs 50,000 — Enhanced Due Diligence trigger
KYC_THRESHOLD	Rs 10,000 — KYC re-verification trigger
STRUCTURING_BANDS	[(9000-9999), (49000-49999), (99000-99999)] — structuring detection ranges
LARGE_AMOUNT_MULTIPLIER	10 — flag if amount > 10× personal average
VELOCITY_BURST_THRESHOLD	3 — more than 3 transactions per hour is suspicious
DEFAULT_ML_WEIGHT / RAG_WEIGHT	0.40 / 0.60 — ML gets 40%, RAG gets 60% in fusion
DISAGREEMENT_THRESHOLD	30 — score gap > 30 triggers conservative max mode
COLD_START_PENALTY	Added to fusion score for customers with < 15 transactions
LOW_CONFIDENCE_FLAG_FLOOR	31 — minimum score when RAG confidence < 0.65

13.2 PRA Constants (pattern_agent/constants.py)

SEQUENCE_LENGTH	30 — transactions per BiLSTM input sequence
FEATURE_DIM	17 — actual features produced by TMA (PDF spec says 15, corrected)
BILSTM_HIDDEN_SIZE	64 — hidden units per direction
POLLER_INTERVAL_MS	500 — poll interval in milliseconds
RAG_L3_SIM_THRESHOLD	0.60 — minimum cosine similarity for typology match

13.3 CLA Constants (cla/constants.py)

STR_AUTO_THRESHOLD	85 — RAA score >= 85: auto-file STR

STR_APPROVAL_THRESHOLD	70 — RAA score >= 70: pending approval
CTR_AMOUNT_THRESHOLD	Rs 10,00,000 — auto-file CTR
ADMIN_USERNAME / ADMIN_PASSWORD	Admin credentials for the dashboard login

14. Testing (et-python/tests/)

The project uses pytest with pytest-cov for test coverage. Tests are organized into unit tests per agent module and integration tests for agent handoffs.

14.1 Unit Tests

Test File	Covers
`test_monitoring_agent/test_agent.py`	TMA pipeline orchestration, failure modes
`test_monitoring_agent/test_anomaly_extractor.py`	15 feature extraction correctness
`test_monitoring_agent/test_decision_engine.py`	Score fusion modes, confidence thresholds
`test_monitoring_agent/test_ml_layer.py`	Isolation Forest loading, scoring, fallback
`test_monitoring_agent/test_profile_builder.py`	Behavioural profile computation, cold start
`test_monitoring_agent/test_response_executor.py`	DB write-back correctness
`test_pattern_agent/test_bilstm_model.py`	BiLSTM forward pass, output shapes
`test_pattern_agent/test_sequence_builder.py`	Matrix construction, zero-padding
`test_pattern_agent/test_pra_agent.py`	PRA pipeline, atomic claiming
`test_pattern_agent/test_pattern_scorer.py`	Verdict mapping, score formula
`test_raa/test_score_engine.py`	60/40 fusion, tier floors
`test_raa/test_tier_engine.py`	T1-T4 classification, CRITICAL demotion
`test_raa/test_regulatory_engine.py`	CTR/STR threshold checks
`test_aba/test_account_controller.py`	Freeze/unfreeze logic
`test_aba/test_case_manager.py`	Fraud case creation
`test_aba/test_gateway_controller.py`	Verdict to gateway action mapping
`test_aba/test_notification_engine.py`	Notification dispatch
`test_cla.py`	STR assembly, PDF generation, status determination
`test_otp_service.py`	OTP generation, hashing, expiry
`test_payment_service.py`	Payment validation, processing

14.2 Integration Tests

Integration tests in tests/integration/ cover the full agent-to-agent handoffs:

- test_full_pipeline.py — End-to-end from payment to CLA filing
- test_payment_flow.py — Payment creation, TMA scoring, commit/hold/reject
- test_tma_pra_handoff.py — TMA → PRA queue + processing
- test_pra_raa_handoff.py — PRA completion → RAA pickup
- test_raa_aba_handoff.py — RAA action package → ABA execution

```
# Run tests
cd et-python
pytest                          # all tests
pytest --cov                    # with coverage report
pytest tests/unit/              # unit tests only
pytest tests/integration/       # integration tests only
```

15. Setup & Running

15.1 Backend Setup

```
# 1. Install Python dependencies
pip install -r requirements.txt

# 2. Configure MySQL in et-python/db.py
#   Change 'Enter Your Password' to your MySQL root password
#   Create database: CREATE DATABASE fraud_detection_test;

# 3. Train BiLSTM model (if bilstm_v1.pt is missing)
python et_service/pattern_agent/train_bilstm.py --mode=synthetic

# 4. Run Flask backend (from et-python/)
python app.py
# Server starts on http://localhost:5000
# All agents start automatically (TMA, PRA, RAA, ABA, CLA)
```

15.2 Frontend Setup

```
# From et-web/
npm install
npm run dev
# Frontend starts on http://localhost:5173
```

15.3 Python Dependencies

Flask==3.0.2	Web framework
Flask-CORS==4.0.0	Cross-origin resource sharing
mysql-connector-python==8.3.0	MySQL database driver
bcrypt==4.1.2	Password hashing
scikit-learn==1.5.1	Isolation Forest ML model
joblib>=1.3.2	Model serialization

numpy>=1.24.0,<2.0	Numerical computing
chromadb>=0.4.0	Vector database for RAG
sentence-transformers>=2.2.0	BERT embeddings (all-MiniLM-L6-v2)
torch (PyTorch)	BiLSTM model (implicit dependency)
reportlab	PDF generation for STR/CTR filings